



9784822296216



1922055022006

ISBN978-4-8222-9621-6
C2055 ¥2200E

定価 [本体2200円+税]
発行 日経BP社
発売 日経BPマーケティング

ドンドン身に付く、スラスラ書ける
書き込み式 **改訂新版**

SQLのドリル

Chapter ① イントロダクション

Chapter ② 基本的な検索テクニックを理解しよう

Chapter ③ より複雑な問い合わせを行おう

Chapter ④ データの登録／更新／削除を行おう

Chapter ⑤ データベース構造を操作しよう

ドンドン身に付く、スラスラ書ける

書き込み式SQLのドリル

改訂新版

山田 祥寛 著

日経BP
9621

ドンドン身に付く、スラスラ書ける
書き込み式 **改訂新版**

SQLのドリル

山田 祥寛 著

計算だって漢字だって
手で書いて覚えませんでしたか？
SQLだって手で書いたほうが
早く着実に身に付きます！

本書の特徴

- MySQL、PostgreSQL、SQLite、SQL Server、Oracleに対応しています
- 解答欄に直接書き込むことで、手を動かしながら覚えられます
- サンプルを使って実際に操作しながら学習を進められます

日経BP社



「はじめに

本書で
とで、実践
います。と
識を取得
と読み解
です。

小学校
漢字ド
ました。お
今さら
返し同じ
動かす
一番の

本書はソシムより刊行されていた「書き込み式 SQL のドリルードン身につく、スラスラ書ける」の内容を一新、再構成したものです。

— 本書の内容は参照用としてのみ使用されるべきものであり、予告なしに変更されることがあります。また、日経 BP 社がその内容を保証するものではありません。本書の内容に誤りや不正確な記述がある場合も、日経 BP 社は一切の責任を負いません。

— 本書に記載されている内容の運用によって、いかなる損害が生じても、日経 BP 社および著者は責任を負いかねますので、あらかじめご了承ください。

本書に記載している会社名および製品名は、各社の商標または登録商標です。なお、本文中に TM、® マークは明記しておりません。

本書の例題または画面で使用している会社名、氏名、他のデータは、一部を除いてすべて架空のものです。

本書の無断複写・複製（コピー等）は著作権法上の例外を除き、禁じられております。購入者以外の第三者による電子データ化および電子書籍化は、私的使用を含め一切認められておりません。

はじめに

本書では、とにかく SQL を繰り返し書くことで、実践力を身に付けることを目的としています。どんな勉強でも同様ですが、ある知識を習得するために、文献や資料をただ漫然と読み解くだけでは（多くの場合に）不十分です。

小学校の頃を思い出してみてください。

漢字ドリルがありました。計算ドリルがありました。あれと同じです。

今さらドリル、と思わないでください。繰り返し同じ（ような）問題を解き、自分の手を動かすこと、それは確かな知識を身に付ける一番の近道です。

本書では、皆さんにできるだけ「自分で」書き込んでいただくために余白を多く用意しています。この余白を真っ黒に埋め尽くすつもりで、まずはとにかく繰り返し書いてください。書き込んだ一文字一文字がきっと皆さんの知識の血肉になるでしょう。

また、本書では実際に SQL 命令を記述する際に「書き順」を強く意識しています。SQL とは Structured Query Language というその名の通り（今は略語ではなく単なる SQL とされていますが）、「構造化」された言語です。書き順を学ぶということは、SQL の構造を意識することでもあります。漫然と自己流で記述するのではなく、書き順を守ることで、自然と構造を理解することにもつながり、それは将来、より複雑な SQL を記述する場合の思考的な手がかりにもなるはずです。

本書が、SQL を学ぶすべての方の最初の一步となることを祈ります。

なお、本書はあくまで SQL の基本的な記述方法を習得することを目的とするもので、SQL の構文を製品横断的に網羅するものではありません。個々のデータベース製品固有の機能や記述については、専門の書籍を併読することを強くお勧めします。



本書に関するサポート情報は、以下の URL で公開しています。Q & A 掲示板はじめ、本書に関する FAQ 情報、オンライン公開記事など、タイムリーな情報を充実した内容でお送りしておりますので、あわせてご利用ください。

<http://www.wings.msn.to/>

最後にはなりましたが、タイトなスケジュールの中、著者の無理を調整いただいた日経 BP 社の編集諸氏、傍らで原稿管理・校正作業などの制作をアシストしてくれた妻の奈美、両親、関係者ご一同に心から感謝の意を表します。

2013 年 3 月吉日 山田 祥寛

イントロダクション

1

0-1	データベースの基本を理解しよう	2
•	データベースは身近な「床下の力持ち」	2
•	どうしてデータベースが必要なの？	3
•	データベースにはどんな種類があるの？	5
0-2	リレーショナルデータベースの基本用語を理解しよう	8
•	データベースの構造	8
•	カラムと制約条件	9
•	カラムは適切な単位に分割しよう	12
•	テーブル設計には欠かせない「正規化」	13
0-3	本書を利用するための準備	18
•	MySQL 5.6 (Windows 版) のインストール方法	18
•	MySQL monitor の利用方法	24
0-4	本書の使い方	30
•	配布サンプルについて	33
•	本書で使用しているサンプルデータベース	34
•	共通テーブル	34
•	アンケート集計アプリケーション	35
•	図書館貸出しアプリケーション	35
•	勤怠管理アプリケーション	36
•	売り上げ管理アプリケーション	37
•	電子商店街アプリケーション	38
•	サイトアクセスログ管理アプリケーション	39
•	書籍の目次管理アプリケーション	39

基本的な検索テクニックを理解しよう

41

1-1	テーブルからすべての列を取り出したい SELECT 命令	42
1-2	テーブルから特定の列を取り出したい SELECT 命令	46
1-3	重複した行をあらかじめ取り除きたい DISTINCT キーワード	51
1-4	条件に合致したレコードだけを取り出したい WHERE 句	55
1-5	あいまい条件に合致したレコードだけを取り出したい LIKE 演算子	63
1-6	複数条件に合致したレコードだけを取り出したい 論理演算子	70
1-7	特定の列を使って行を並べ替えたい ORDER BY 句	78
1-8	特定範囲のレコードのみを取り出したい LIMIT 句	86
1-9	特定の列を使ってデータをグルーピングしたい GROUP BY 句	93
1-10	列に別名を指定したい AS キーワード	100
1-11	文字列／日付データの加工を行いたい 文字列／日付関数	107
1-12	グルーピング結果に対して条件を付与したい HAVING 句	114
1-13	条件によって値を変更したい CASE 演算子	121

より複雑な問い合わせを行おう 129

- 2-1 内部結合で複数テーブルのデータを結合したい 130
INNER JOIN...ON 句
- 2-2 外部結合で複数テーブルのデータを結合したい 139
RIGHT JOIN...ON 句 / LEFT JOIN...ON 句
- 2-3 自己結合で同一テーブル内のデータを結合したい 149
JOIN 句 + 別名定義
- 2-4 3 つ以上のテーブルを結合したい (結合の応用) 156
JOIN 句の入れ子
- 2-5 ある問い合わせ結果に基づいて、検索したい 164
サブクエリ
- 2-6 複数テーブルから取り出したデータを統合したい 174
UNION 句

データの登録 / 更新 / 削除を行おう 185

- 3-1 新規のデータを挿入したい 186
INSERT 命令
- 3-2 列指定で新規のデータを挿入したい 191
INSERT 命令
- 3-3 検索結果を他のテーブルに挿入したい 198
INSERT 命令 / サブクエリ
- 3-4 既存データを更新したい 205
UPDATE 命令
- 3-5 特定の条件に合致するデータを更新したい 210
UPDATE 命令
- 3-6 ある問い合わせ結果に基づいて、更新したい 216
UPDATE 命令 / サブクエリ
- 3-7 既存データを削除したい 223
DELETE 命令
- 3-8 ある問い合わせ結果に基づいて、削除したい 231
DELETE 命令 / サブクエリ

データベース構造を操作しよう 237

- 4-1 テーブルを作成したい 238
CREATE TABLE 命令
- 4-2 新規にインデックスを作成したい 246
CREATE INDEX 命令
- 4-3 既存テーブルに列や制約条件を追加したい 254
ALTER TABLE 命令
- 4-4 既存テーブル上の列や制約条件を変更したい 260
ALTER TABLE 命令

マスタ・プラクティス解答集 265

- Chapter 1 基本的な検索テクニックを理解しよう 266
- 1-1 テーブルからすべての列を取り出したい SELECT 命令 266
- 1-2 テーブルから特定の列を取り出したい SELECT 命令 266
- 1-3 重複した行をあらかじめ取り除きたい DISTINCT キーワード 267
- 1-4 条件に合致したレコードだけを取り出したい WHERE 句 267
- 1-5 あいまい条件に合致したレコードだけを取り出したい LIKE 演算子 269
- 1-6 複数条件に合致したレコードだけを取り出したい 論理演算子 270
- 1-7 特定の列を使って行を並べ替えたい ORDER BY 句 271
- 1-8 特定範囲のレコードのみを取り出したい LIMIT 句 272
- 1-9 特定の列を使ってデータをグルーピングしたい GROUP BY 句 273
- 1-10 列に別名を指定したい AS キーワード 274
- 1-11 文字列 / 日付データの加工を行いたい 文字列 / 日付関数 275
- 1-12 グルーピング結果に対して条件を付与したい HAVING 句 277
- 1-13 条件によって値を変更したい CASE 演算子 278

Contents

Chapter2 より複雑な問い合わせを行おう	280
2-1 内部結合で複数テーブルのデータを結合したい INNER JOIN...ON 句	280
2-2 外部結合で複数テーブルのデータを結合したい	281
RIGHT JOIN...ON 句 / LEFT JOIN...ON 句	
2-3 自己結合で同一テーブル内のデータを結合したい JOIN 句+別名定義	282
2-4 3 つ以上のテーブルを結合したい (結合の応用) JOIN 句の入れ子	283
2-5 ある問い合わせ結果に基づいて、検索したい サブクエリ	285
2-6 複数テーブルから取り出したデータを統合したい UNION 句	286
Chapter3 データの登録 / 更新 / 削除を行おう	288
3-1 新規のデータを挿入したい INSERT 命令	288
3-2 列指定で新規のデータを挿入したい INSERT 命令	289
3-3 検索結果を他のテーブルに挿入したい INSERT 命令 / サブクエリ	290
3-4 既存データを更新したい UPDATE 命令	291
3-5 特定の条件に合致するデータを更新したい UPDATE 命令	292
3-6 ある問い合わせ結果に基づいて、更新処理したい	293
UPDATE 命令 / サブクエリ	
3-7 既存データを削除したい DELETE 命令	294
3-8 ある問い合わせ結果に基づいて、削除したい	295
DELETE 命令 / サブクエリ	
Chapter4 データベース構造を操作しよう	297
4-1 テーブルを作成したい CREATE TABLE 命令	297
4-2 新規にインデックスを作成したい CREATE INDEX 命令	298
4-3 既存テーブルに列や制約条件を追加したい ALTER TABLE 命令	299
4-4 既存テーブル上の列や制約条件を変更したい ALTER TABLE 命令	300
索引	302

Chapter

0

イントロダクション

本書のテーマである SQL (Structured Query Language[※]) は、データベース (その中でも、主にリレーショナルデータベース) を扱うための標準的な問い合わせ言語です。

本書では、この SQL の基本的な記法をドリル形式で学習していくわけですが、SQL の学習を始めるに先立って、まずは SQL が操作する対象であるデータベースについて概要を理解しておくことにしましょう。

[※] 注) SQL は当初、Structured Query Language (構造化問い合わせ言語) の略であるとされてきましたが、昨今では、特に略語ではなく、それそのものを固有名詞と捉えるのが一般的です。

0-1 データベースの基本を理解しよう

SQL は、リレーショナルデータベース (RDB: Relational DataBase) に対して問い合わせを行うための標準的な言語です。SQL について学ぶ前に、本節ではまずデータベースの基本を理解しておくことにしましょう。

データベースは身近な「床下の力持ち」

データベースとは、その名の通り、なにかしらの情報 (データ) を蓄積した集合体です。ただし、ただ無作為に蓄積すれば良いというものではありません。あとから検索/分析することを目的に、ある一定のルールに基づいて蓄積した情報のことを**データベース (DB: DataBase)** と言います。また、データベースを管理するための専用ソフトウェアのことを**データベース管理システム (DBMS: DataBase Management System)** と言います (ただし、本書では特別に限定することなく「データベース」といった場合、「データベース管理システム」を含むものとします)。

もっとも、データベースというと、「なんか難しそうなんだよね」「そもそも自分の生活とは無関係なんじゃない?」などと考えている人も多いようです。あるいは、本書を手にとって「未知の世界へ乗り出すんだ」と意気込んでいる方も少なくないかもしれません。しかし、最初に強調しておきたいのは、データベースとはなにも「特別なものではない」ということです。私たちの周りには多くのデータベースが存在しており、多くの方が日常的にデータベースを利用しているはずです。

たとえば、インターネット上でなにか探し物をしたいという場合に、今や Google や Yahoo! のような検索エンジンは欠かせないものです。SQL に関する書籍を検索したいならば、検索キーワードに「SQL 書籍」などを入力して [検索] ボタンをクリックします。すると、キーワードに合致した Web ページの一覧が即座に表示されるはずです。

また、近頃では Amazon (<http://www.amazon.co.jp/>) や 楽天 (<http://www.rakuten.co.jp/>) のような、いわゆる電子コマース (EC) サイトでさまざまな商品を購入することも珍しいことではなくなりました。このようなサイトで、私たちはまず欲しい商品を検索し、気に入った商品があればこれを買いたいカゴに登録します。買い物を楽しんだ後は、送付先情報やクレジットカードの番号などを入力し、注文を確定します。注文した品物が届くまでは、発送予定日などを検索したり、注文の変更/取消などを行うかもしれませんし、過去の購入履歴を検索するかもしれません。

はたまた、皆さんは旅行に行くときに飛行機や新幹線の席を窓口で購入するでしょう。このような場合に、駅員さんはコンピュータを確認して、即座に空席情報を確認 (検索)

してくれるはずです。満席であれば、異なる条件で空席を探してもらうこともできますし、空席が残っていれば予約し、切符の発券をしてくれることでしょう。

これら情報のすべては、データベース上で蓄積/管理され、データベースの機能を利用することで即座に取り出すことができるわけです。データベースがプログラマなどコンピュータの世界に関わる人のためだけのものではなく、日常生活の中でなくてはならないものであることがお分かりになるのではないのでしょうか。

どうしてデータベースが必要な?

もっとも、少しでもプログラムをかじったことがある方ならば、データを出し入れするだけなら、「わざわざデータベースを使わなくても、テキストファイルに直接書き込めばいいんじゃない?」、そう思われる方もいるかもしれません。なるほど、テキストファイルならば、特別な知識も特別なソフトウェアも要りませんから、一見して手軽のようにも思えます。しかし、データベースを利用することには、そんな目先の手軽さからは得られない多くのメリットがあるのです。

① データの管理をブラックボックス化

テキストファイルなどにデータを格納する場合、「どのようにデータを保存 (参照) するのか」は、すべてアプリケーション側で決める必要があります。

実装の方法によっては、データ項目に変更があったり、増減したりした場合に、読み書きのプログラムそのものに影響が出ることもあるでしょう。

また、データが 10 件 100 件レベルならばいざ知らず、1,000 件 10,000 件と増大した場合、アプリケーションによっては処理速度が著しく低下する場合もあるでしょう (そして、処理速度が低下しにくいアプリケーションを構築すること自体、とても難しいことです)。

しかし、データベースを利用することで、アプリケーション側ではデータの出し入れについてなんら意識する必要はなくなります。データの格納から更新、削除、検索までをすべてデータベースに任せることができるので、開発生産性が向上します。

② 大切なデータを安全管理

言うまでもなく、データとは重要な「資産」です。たとえば、あなたがパソコン上で管理しているアドレス帳、これが明日突然消えてしまったらどうでしょう。あるいは、更新している途中にコンピュータの障害で壊れてしまったら。

データベースはこのようなトラブルから大切なデータを守るためのさまざまな機能を提供してくれます。バックアップ機能をはじめ、更新の途中で障害が発生した場合にデータの整合性を保つ機能など、データを安全に管理するための諸機能を、データベースは用意してくれます。

③ 共通言語 SQL でカンタン操作

そして、本書のテーマである **SQL** の登場です。現在の代表的なデータベースのほとんどは、問い合わせのための標準的な言語として SQL に対応しています。SQL はデータベース製品ごとに多少の「方言」こそあるものの、ANSI や JIS で規定された国際標準規格に準拠していますので、使用するデータベース製品を変更しても、アプリケーションへの影響は最小限に抑えることができます。

また、SQL 自体はデータ処理に特化した言語で、構文も英語のそれによく似ていますので、習得も容易なのが特長です。

SQL は表のように、大きく 3 つに分類することができます。このうち、DCL はロックタイプや権限型などデータベース製品ごとの固有の仕様差が大きいことから、本書では扱っていません。詳細は、データベース製品個々のドキュメントを参照してください。

本書では **DML** と **DDL**、その中でも最も使用する機会も多く、さまざまなオプションを持っている **SELECT** 命令を中心に解説していきます。

● SQL の分類と主な命令

分類	命令	概要
DML (Data Manipulation Language : データ操作言語)		
	SELECT	データの検索
	INSERT	新規データの挿入
	UPDATE	既存データの更新
	DELETE	既存データの削除
DDL (Data Definition Language : データ定義言語)		
	CREATE	データベースやテーブル、インデックスの作成
	ALTER	データベースやテーブル、インデックスの更新
	DROP	データベースやテーブル、インデックスの破棄
DCL (Data Control Language : データ制御言語)		
	GRANT	ユーザ権限の付与
	REVOKE	ユーザ権限の剥奪
	BEGIN	トランザクションの開始
	COMMIT	トランザクションの確定
	ROLLBACK	トランザクションのキャンセル

NOTE

本稿執筆時(2013年1月)における最新の SQL 仕様は、2011年に制定された SQL 2011 です。

SQL 仕様は、1989年の SQL89 でその大枠が決定され、次の SQL 92 で SQL 89 で曖昧だった仕様を明確に定義しています。その後、SQL 99 ではオブジェクト指向のサポート、SQL2003 では SQL/XML、SQL2011 では INSTEAD OF トリガーなどと、より高度な機能が追加されています。

このように、データベースとは、大切なデータ資産を「確実に」、かつ「高速に」処理するためには欠かせないソフトウェアです。ちなみに、データベースの中でも、特にネットワーク経由で複数のユーザ間でデータを共有する目的で用いられるデータベースのことを **データベースサーバ** と呼びます。

データベースサーバは、データベースの基本的なデータ管理機能に加えて、複数ユーザが同時アクセスした場合に更新の矛盾が発生しないようにするための「**排他制御**」や、データやユーザ単位にアクセスの可否を設定するための「**セキュリティ機能**」などを備えています。

データベースにはどんな種類があるの?

ここまで「データベース」とひと括りで紹介してきましたが、実はデータベースにはさまざまな形態があります。

たとえば、ある1つの情報を1枚のカードで管理する **カード型データベース** や、データとデータ同士が互いに親子関係で表現できる **ネットワーク型データベース**、ネットワーク型の亜流として、1つの子供が常に1つの親しか持つことができない **ツリー型データベース** などです。しかし、現在の主流のデータベースと言え、なんといっても **リレーショナルデータベース (RDB)** でしょう。

リレーショナルデータベースはすべてのデータを2次元の表形式で表現し、関係する項目同士に **リレーションシップ** (関係) を設定することで複雑なデータも表現することができます。カード型やネットワーク型などのデータベースに比べて、人間が直感的に理解しやすいことから急速に普及し、現在ではただ単に「データベース」といった場合には、「リレーショナルデータベース」のことを指すと思って間違いありません。

昨今では Java や C# のようなオブジェクト指向言語の普及に伴って、アプリケーション側との親和性に優れた「**オブジェクト指向データベース**」や、キー/値のセットでデータを管理するシンプルさからスケーラビリティに優れる「**Key-Value 型データベース**」などが話題に上ることも多くなってきましたが、現時点ではまだまだ発展途上の分野であり、これらは今後の展開に期待したいところです。

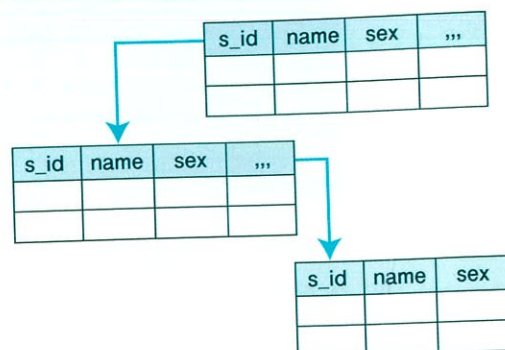
0 イントロダクション

カード型データベース



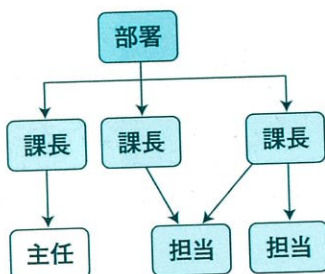
1レコードを1枚のカードとして管理。
古い型のデータベースで、現在はほとんど見られない

リレーショナルデータベース



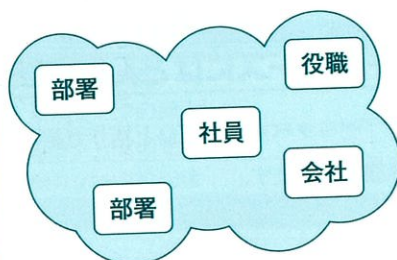
1つの対象を2次元のテーブル表で管理。
テーブル内で設定されたキー同士を設定することで
複雑なデータの表現も可能

ネットワーク型データベース



網型データベースとも呼ばれる。
より限定された形態として、1つの子に
対して、1つの親しか持たないツリー
型データベースもある

オブジェクト指向データベース



対象をオブジェクトとして管理。
オブジェクト指向言語との親和性に優れる。
その一形態として、XMLデータベースもある

Key-Value型データベース

Key	Value
001	
002	
003	
...	...

キー／値のセットで
データを管理。
スケーラビリティに
優れており、昨今再び
注目を集めている

以下では、執筆時点で比較的良好に耳にする主要なデータベース製品を挙げておきます。

これだけ機能やサポートしている言語がかなり違う

0-1 データベースの基本を理解しよう

●主要なデータベース製品

製品名	概要
MySQL (http://www.mysql.com)	パフォーマンスに定評があり、世界的にも浸透しているデータベース。パフォーマンス重視であることからバージョン 4.1 までは実装機能がやや限定されていたが、5.0以降で大幅に機能強化している。Windows / Unix など主要なプラットフォームに対応
PostgreSQL (http://www.postgresql.org)	オープンソースで提供される高機能なデータベース。商用バージョンとして PowerGRES も提供。バージョン 7.x までは Unix 系プラットフォーム中心の対応を進めてきたが、バージョン 8.0 以降では Windows 環境にネイティブ対応した
SQLite (http://www.sqlite.org/)	アプリケーションへの組み込み用途が中心の軽量データベース。スクリプト言語である PHP、Python などが標準サポートしているほか、Android でも標準ライブラリとして採用される
Oracle Database (http://www.oracle.com/jp)	Oracle 社開発の商用データベースサーバ。ビジネスシーンにおいて古くから多く採用され、現在日本で最もよく利用されている RDB の 1 つ
SQL Server (http://www.microsoft.com/ja-jp/sqlserver/)	Microsoft 社開発の商用データベースサーバ。Oracle に比べると後発ではあるが、小規模から大規模システムまで幅広いラインナップが提供されており、GUI 管理ツールも充実しているのが特長。Windows プラットフォームで強い人気を誇る
DB2 (http://www-06.ibm.com/software/jp/data/)	IBM 社開発の商用データベースサーバ。Windows / Unix はじめ、主要なプラットフォームに対応し、古くから多くのシステムで採用されている
Access (http://office.microsoft.com/ja-jp/access/)	個人用途主体のデータベースソフトウェア。GUI ツールが充実しており、初心者にも使いやすいのが特長。標準的なインターフェイスである ODBC (Open DataBase Connectivity) を利用することで、他データベースのフロントエンドとして利用することも可能 ([Note] 参照)

NOTE

Access の詳細な設定方法については、以下の記事などが参考になります。

「データベース・サーバのフロントエンドとして Access を利用する」

<http://www.atmarkit.co.jp/fwin2k/win2ktips/346dbaccess/dbaccess.html>

0-2 リレーショナルデータベースの基本用語を理解しよう

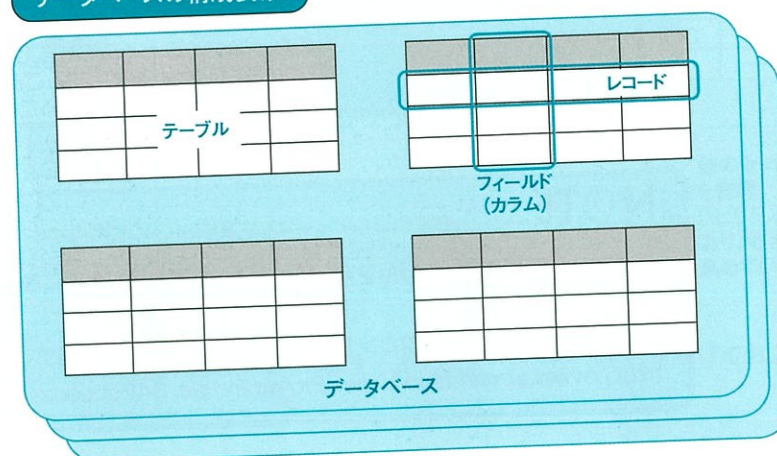
リレーショナルデータベースを扱っていく中では、さまざまなキーワードが登場します。どこかで聞いたことのある単語、聞きなれない単語、さまざまあるかもしれませんが、学習に先立って、まずはこれら基本キーワードをおさえておくことにしましょう。

データベースの構造

繰り返しますが、データベースとはなにかしらのデータを蓄積した集合体です。そして、リレーショナルデータベースにおいては、関連するデータをすべて、**テーブル**と呼ばれる2次元の表に格納します。テーブルとは、Microsoft Excel などの表計算アプリケーションで扱うワークシートのようなものだと思っていただいても良いかもしれません。やや雑駁とした説明ですが、データベースとはテーブルの集合体なのです。

テーブルの横1列の行のことを**レコード**と言い、レコードに含まれる個々の項目（縦の列）のことを**カラム**または**フィールド**と言います。

データベースの構成要素



NOTE

データベースには、実際にはテーブル以外にも、仮想的な表を表す「ビュー (P.170)」や一連のSQL手続きをまとめた「ストアドプロシージャ」、連番を管理するための「シーケンス」、挿入、更新、削除時の処理を規定する「トリガー」などのオブジェクトが含まれます。

もっとも、表計算アプリケーションとデータベースとは決定的に異なる点もあります。それは、データベースではそれぞれのカラムにどのようなデータをセットするのかをあらかじめ決めておく必要があるという点です。

たとえば、アドレス帳を管理したテーブルに「誕生日」という列があったとします。あるレコードでは「1999年12月04日」という文字列でセットされているのに、あるレコードでは「1985/08/05」のような形式でデータがセットされているとしたらどうでしょう。また、「子供の数」という列があったとしたとき、あるレコードには「なし」と文字列がセットされており、あるレコードには「0」とセットされているとしたら？

このようなデータの不揃いは、検索時に意図したデータの抽出ができなくなってしまう原因にもなります。

そこで、データベースではテーブルを用意する際に、それぞれのカラムに**データ型**をセットしておく必要があります。「誕生日」列は日付型、「子供の数」列は非負の整数型というようにあらかじめ設定しておくことで、意図しないデータがセットされそうになった場合には、データベースが警告を発してくれるというわけです。

利用可能なデータ型や名称は、使用しているデータベース製品によって異なりますので、P.10の表には本書で扱うデータベースにおける主要なデータ型を挙げてきます。ただし、同名の型であってもデータベースによって扱える範囲が異なっていたり、そもそも型名が異なるものなどありますので、詳細はデータベース製品個々のドキュメントを参照するようにしてください。

NOTE

一般的に、カラム単位に厳密なデータ型を設定できるのがRDBの特徴ですが、例外的にデータ型をサポートしないRDBもあります。サーバサイド実行環境であるPHP 5に標準実装されたことで話題を呼んだSQLite (<http://www.sqlite.org/>) がそれです。SQLiteでは、データ型を指定することも可能ですが、データを挿入したり更新したりする際に、データ型はチェックされませんので、注意してください。

カラムと制約条件

カラムの性質を決めるのは、データ型ばかりではありません。その他にもデータの内容を規定するためのさまざまな条件を定義することが可能です。

たとえば、書籍情報テーブルに含まれる「ISBNコード（本書では10桁の旧規格のコードを使います）」列を考えてみましょう。ISBNとはInternational Standard Book Numberの略で、世の中に流通している書籍を一意に表すための番号のことを言います。ISBNコードは、1つの書籍に対して必ず1つ割り振られているもので、重複するはずの

ない番号です。このようなカラムのことを**主キー (PRIMARY KEY)**といいます。主キーはテーブル内のレコードを一意に特定するためのキーで、テーブル内に常に1つしか設定することができません(複数のカラムから構成される複合キーは可)。逆に、主キーは1つのテーブルに対して必ず設定するべきです。主キーの重複したレコードをテーブルに登録することはできません。

●各データベースにおける主要なデータ型 (n: 全体の桁数、d: 小数点以下の桁数)

	MySQL	PostgreSQL	SQLite	Oracle	SQL Server
数値型	TINYINT SMALLINT MEDIUMINT INT BIGINT FLOAT DOUBLE(n,d) DECIMAL(n,d)	BIT SMALLINT INT BIGINT FLOAT4 FLOAT8 NUMERIC(n,d) MONEY	INTEGER REAL	INT SMALLINT NUMBER (n,d)	BIT TINYINT SMALLINT INT BIGINT FLOAT(n) REAL SMALLMONEY MONEY
文字列型	CHAR(n) VARCHAR(n) TINYTEXT TEXT MEDIUMTEXT LONGTEXT	CHAR(n) VARCHAR(n) TEXT	TEXT	CHAR(n) NCHAR(n) VARCHAR2(n) NVARCHAR2(n) LONG CLOB NCLOB	CHAR(n) NCHAR(n) VARCHAR(n) NVARCHAR(n) TEXT NTEXT
日付型	DATETIME DATE TIME TIMESTAMP YEAR	DATE TIME TIME WITH TIME ZONE TIMESTAMP INTERVAL	TEXT REAL INTEGER	DATE TIMESTAMP (n)	DATE TIME DATETIME2
バイナリ型	TINYBLOB BLOB MEDIUMBLOB LONGBLOB	(OID)	BLOB	BLOB	IMAGE

主キーと外部キー

書籍情報テーブル			
isbn	title	price	...
4-0010-0000-0	ハムスターの観察	900	...
4-7980-0945-8	PHP5 サンプル集	3000	...
4-8833-0000-2	SQL プチブック	1600	...
4-7981-0722-0	XML 辞典	3300	...



主キー：レコードを1つ(一意)に特定するためのキー
重複は許さない



著者情報テーブル	
isbn	author
4-0010-0000-0	山田愛子
4-0010-0000-0	佐藤一郎
4-7980-0945-8	佐藤一郎
4-8833-0000-2	川中智樹
4-7981-0722-0	山田祥寛

主キーと外部キーとの
関連付けによって、
複数のテーブルを
1つにもできる

外部キー：主キーを参照するためのキー
重複は OK。しかし、主キーに対応していること！

isbn	title	author	price	...
4-0010-0000-0	ハムスターの観察	山田愛子	900	...
4-0010-0000-0	ハムスターの観察	佐藤一郎	900	...
4-7980-0945-8	PHP5 サンプル集	佐藤一郎	3000	...
4-8833-0000-2	SQL プチブック	川中智樹	1600	...
4-7981-0722-0	XML 辞典	山田祥寛	3300	...

主キーを参照するキーのことを**外部キー (Foreign Key)**といいます。

たとえば、著者情報テーブルのようなテーブルを想定してみましょう。著者情報テーブルは、書籍情報テーブルに登録されている書籍に対応する著者情報を登録するテーブルで、1冊の書籍に対して複数の著者が存在する場合もあるものとします。このような場合、著者情報テーブルは「どの書籍の著者なのか」を表すために「ISBN コード」列を持つ必要があります。著者情報テーブルの「ISBN コード」列は、当然、書籍情報テーブルの「ISBN コード」列の内容と対応関係にある必要があります。

RDB では、このようにテーブル間の主キーと外部キーとで対応関係を持たせることで、互いのテーブルを関連付けられます。このような関連付けのことを**リレーションシップ**と言い、主キーと外部キーとの対応関係に関する制約条件のことを**参照制約**といいます。

参照制約とは、主キー、外部キーが必ず対応関係を持っていることを保証するチェックのことです。上の例の場合、外部キーである著者情報テーブルの「ISBN コード」列では、主キーである書籍情報テーブルにない値をセットすることはできません。また、著者情報テーブル(外部キー)で参照している値を、書籍情報テーブルで更新/削除することはできません。

参照制約

書籍テーブル			
isbn	title	price	...
4-0010-0000-0	ハムスターの観察	900	...
4-7981-0959-2	PEAR 入門	3000	...
4-7980-0945-8	PHP5 サンプル集	3000	...
4-8833-0000-2	SQL プチブック	1600	...
4-7981-0722-0	XML 辞典	3300	...

→ 外部キーで参照している主キーを消すのは不可

著者情報テーブル	
isbn	author
4-0010-0000-0	山田愛子
4-7981-0959-2	佐藤一郎
4-7980-0945-8	佐藤一郎
4-8833-0000-2	川中智樹
4-7981-0722-0	山田祥寛

主キー

isbn	author
0-0000-0000-0	山田太郎

挿入
主キーにない値は外部キーとして参照できない

外部キー

その他にも、**非 NULL 制約**（値が空であることを許さない）や **UNIQUE 制約**（重複した値を許さない）などもありますが、これらについては、Chapter4 で改めて紹介したいと思います。

データベースでは、このようにカラム単位にさまざまな条件を付与しておくことで、想定しないデータが混入することを防ぐことができます。厳密な条件を付与することは、検索時にも意図したデータを取り出しやすくするという観点から非常に重要なことです。

🔧 カラムは適切な単位に分割しよう

データベースとしての機能ではありませんが、テーブルを設定する場合、カラムを適切な単位で分割することは重要です。たとえば、アドレス帳テーブルを考えてみてください。

住所を1つのカラムで管理していたとしたらどうでしょう。「千葉県小金井市静波 0-0-00」のように登録する人もいれば、「小金井市静波 0-0-00」のように県名を省略して登録する人もいるかもしれません。この場合、たとえば千葉県の人だけを後から検索したいと思っても、後者の住所を取り出すことはできません。

そこで、今度は住所を「県名」「市町村名」「番地、マンション名」という3つのカラムで管理することにします。すると、「県名」列には必ず県名が指定されているはずですから、県ごとの抽出が可能になります。

テーブル上でカラムを設定する場合には、「どのような単位（キー）で検索するか」ということをあらかじめ想定して設計すると、あとからデータの抽出がラクに行えるはずです。

🔧 テーブル設計には欠かせない「正規化」

突然ですが、以下のようなテーブルがあったとしたらどうでしょう。

●非正規化状態のテーブル（注文書テーブル）

注文番号	顧客番号	顧客名	顧客住所	商品番号 1	商品名 1	単価 1	購入数 1	商品番号 2	...
P00001	USR001	山田奈美	千葉県小金井市 福山 0-0-00	P0001	どっさりカイロ	300	10	P0002	Windows Versa XP
P00002	USR002	薄井浩	静岡県鎌ヶ谷市 静波 9-8-98	P0001	どっさりカイロ	300	5		
P00003	USR003	日尾晃子	埼玉県新田原市 三次 1-1-11	P0002	Windows Versa XP	21500	2	P0003	サンタのカッコ
P00004	USR001	山田奈美	千葉県小金井市 福山 0-0-00	P0010	豪華おせち	15000	1		

表計算ソフトで管理しているデータならばこれでも問題ありませんが、RDBに登録するデータとしては、あまり良い例とはいえません。というのも、上のテーブルにはいくつかの問題点があるからです。

- ・顧客名や商品名など重複した情報が多いため、データ量が不必要に大きくなりがち
- ・同様の理由で、商品名や顧客名に変更があった場合に、関係するすべてのレコードを修正しなければならない

このような不具合を解消するために、データを一定の規則に従って分割し、データの重複を取り除くということが行われます。このような手続きのことを**正規化**と言います。正規化は、RDBにおいてデータが増大した場合にも、データの整合性を保つために重要な考え方です。

以下では、上記の注文書テーブルを正規化してみることにしましょう。

STEP 1 第1 正規化

まず正規化の第1段階では、「データ内の繰り返し部分をなくすこと」です。

データ内の繰り返し部分とは、この場合、商品名、単価、購入数です。このようにテーブル内に繰り返し項目を持っているのは好ましいことではありません。たとえば、注文書テーブルのケースならば、1人の顧客が1度に購入する商品の種類が必ずしも決まっているわけではありません。ある顧客は1種類しか買わないかもしれませんが、ある顧客は10種類の商品を1度に購入していくかもしれません。テーブル上に繰り返し項目がある場合、想定される繰り返し上限をあらかじめ設定しておく必要があります。しかし、これはあまりに無駄なことですし、そもそもテーブル上の制約から1度に注文できる商品の種類が制限されてしまうというのは、本末転倒な話でしょう。このような繰り返し項目は別

のテーブルとして分割してしまいます。第1正規化の結果、できたテーブルは以下の通りです。

なお、表内で色の付いている項目は主キーを表すものとします。

●第1正規化後のテーブル

注文番号	顧客番号	顧客名	顧客住所
PO0001	USR001	山田奈美	千葉県小金井市福山 0-0-00
PO0002	USR002	薄井浩	静岡県鎌ヶ谷市静波 9-8-98
PO0003	USR003	日尾晃子	埼玉県新田原市三次 1-1-11
PO0004	USR001	山田奈美	千葉県小金井市福山 0-0-00

▲注文テーブル

注文番号	商品番号	商品名	単価	購入数
PO0001	P0001	どっさりカイロ	300	10
PO0001	P0002	Windows Versa XP	21500	1
PO0002	P0001	どっさりカイロ	300	5
PO0003	P0002	Windows Versa XP	21500	2
PO0003	P0003	サンタのカッコ	5000	1
PO0004	P0010	豪華おせち	15000	1

▲注文明細テーブル

STEP2 第2正規化

次のステップでは、「主キー列以外の列の値が、必ず主キーによって決まる」ようにすることです。ある列の値が特定の列の値によって決まることを、やや難しい言い方をすると**関数従属**と呼びます。

それでは第1正規化を終えた先ほどのテーブルを、第2正規化に見合うように変更してみましょう。まず注文テーブルに注目してみると、こちらは主キーである注文番号が決まると、従属する顧客番号、顧客名、顧客住所は決まります。つまり、既に第2正規化の条件を満たしているということです。次に、購入商品テーブルはというと、一見、主キーである注文番号、商品番号が決まると、残る商品名、単価、購入数が決まるようにも思われます。しかし、よく考えてみてください。商品名、単価は主キーの一部である商品番号だけに従属するものです。

そこで、ここでは商品テーブルとして商品情報を分割します。なお、注文ごとの値引きや特価、価格改定などを考慮すれば、「単価」列は注文明細テーブルに持つべきだという意見もあるかもしれませんが、ここでは単価は固定であることを前提としておきます。

こうしてできたのが、以下のテーブルです。

●第2正規化後のテーブル

注文番号	顧客番号	顧客名	顧客住所
PO0001	USR001	山田奈美	千葉県小金井市福山 0-0-00
PO0002	USR002	薄井浩	静岡県鎌ヶ谷市静波 9-8-98
PO0003	USR003	日尾晃子	埼玉県新田原市三次 1-1-11
PO0004	USR001	山田奈美	千葉県小金井市福山 0-0-00

▲注文テーブル

注文番号	商品番号	購入数
PO0001	P0001	10
PO0001	P0002	1
PO0002	P0001	5
PO0003	P0002	2
PO0003	P0003	1
PO0004	P0010	1

▲注文明細テーブル

商品番号	商品名	単価
P0001	どっさりカイロ	300
P0002	Windows Versa XP	21500
P0003	サンタのカッコ	5000
P0010	豪華おせち	15000

▲商品テーブル

STEP3 第3正規化

最後に、第3正規化では「主キー列以外の列の値が主キーのみによって決まる」ようにする必要があります。第2正規化との区別が少々分かりにくいかもしれませんが、注文テーブルに注目してみてください。

先述したように、注文テーブルは主キーである注文番号によって顧客番号、顧客名、顧客住所は決まります。しかし、よく考えてみてください。顧客名や顧客住所は、注文番号によってのみ決まるわけではなく、実は顧客番号によって決まるのです。

このようなデータは冗長ですので、顧客テーブルとして分割します。その結果できたのが以下のテーブルです。

●第3正規化後のテーブル

注文番号	顧客番号	顧客番号	顧客名	顧客住所
PO0001	USR001	USR001	山田奈美	千葉県小金井市福山 0-0-00
PO0002	USR002	USR002	薄井浩	静岡県鎌ヶ谷市静波 9-8-98
PO0003	USR003	USR003	日尾晃子	埼玉県新田原市三次 1-1-11
PO0004	USR001			

▲注文テーブル

注文番号	商品番号	購入数
PO0001	P0001	10
PO0001	P0002	1
PO0002	P0001	5
PO0003	P0002	2
PO0003	P0003	1
PO0004	P0010	1

▲注文明細テーブル

商品番号	商品名	単価
P0001	どっさりカイロ	300
P0002	Windows Versa XP	21500
P0003	サンタのカッコ	5000
P0010	豪華おせち	15000

▲商品テーブル

いかがですか？これで、正規化のプロセスは完了です。

場合によっては、この後にボイスコード (BC) 正規化、第4正規化、第5正規化などが登場する場合もありますが、基本としては第3正規化まで理解していれば十分でしょう。

もちろん、正規化されたテーブル群は、リレーションシップを辿っていくことで非正規化時の情報を再現できなければなりませんし、またできるはずです。

たとえば、注文番号「PO0001」に注目してみましょう。PO0001に対応する顧客情報と商品番号はそれぞれ顧客テーブル、注文明細テーブルから得られます。注文明細テーブルの商品番号をキーとすることで、商品テーブルから商品情報を得ることができます。確かに元の注文情報が再現できることが確認できました。

●テーブルの結合

注文番号	顧客番号
PO0001	USR001
PO0002	USR002
PO0003	USR003
PO0004	USR001

▲注文テーブル

顧客番号	顧客名	顧客住所
USR001	山田奈美	千葉県小金井市福山 0-0-00
USR002	薄井浩	静岡県鎌ヶ谷市静波 9-8-98
USR003	日尾晃子	埼玉県新田原市三次 1-1-11

▲顧客テーブル

注文番号	商品番号	購入数
PO0001	P0001	10
PO0001	P0002	1
PO0002	P0001	5
PO0003	P0002	2
PO0003	P0003	1
PO0004	P0010	1

▲注文明細テーブル

商品番号	商品名	単価
P0001	どっさりカイロ	300
P0002	Windows Versa XP	21500
P0003	サンタのカッコ	5000
P0010	豪華おせち	15000

▲商品テーブル

冒頭で述べたように、正規化を行う最大の目的は「更新時のデータの整合性を維持する」という点にあります。たとえば、顧客住所に変更があった場合、非正規化状態のテーブルではその顧客が関係するすべての注文情報を修正する必要がありますが、第3正規化のテーブルの場合、該当する顧客の情報1件だけを修正すれば事足りることが分かるはずです。

また、先ほど紹介した「参照制約」を利用することで、商品テーブルにない商品は注文できないなどのガードをかけることも可能でしょう。

正規化は、より複雑な構造のデータを扱うようになった場合、データ量が増大した場合には欠かせないプロセスです。

NOTE

正規化とはあくまでデータの整合性を高めるのが目的で、正規化を行った場合、テーブル結合のオーバーヘッドが発生するために、データ検索のパフォーマンスは低下する可能性があります。もっとも、昨今では、ハードウェアの性能も向上しており、正規化によるパフォーマンス低下の影響はそれほど気にする必要はありませんが、どうしても期待するパフォーマンスが得られないという場合には「非正規化」(冗長化)を行うというアプローチもあります。しかし、あくまで非正規化は最終的な代替策であり、テーブル設計の基本は正規化です。

0-3 本書を利用するための準備

前置きはさておき、早くドリルを始めたい——と思う皆さんは、はやる心をおさえて、もうちょっとだけ辛抱してください。先述したように、SQL はリレーショナルデータベースで標準的に利用可能な言語ですので、ドリルを進めていくにあたっては、自分が使い慣れたデータベース製品を利用してかまいません。

ここでは、まだデータベースに触れるのも初めて、という皆さんのために、本書推奨の環境として MySQL 5.6 のインストール方法や環境設定について紹介しておくことにします。

本書例題は MySQL 5.6 をベースに記述していますので、MySQL を利用することで、よりスムーズに学習を進められるはずです。

なお、PostgreSQL や SQLite、Oracle、SQL Server など、その他のデータベースをご利用の方も心配することはありません。本文中に、各データベースごとの方言についても解説していますので、それぞれのデータベースごとの差異については、こちらの説明を確認してください。

また、配布サンプルには、MySQL だけでなく、PostgreSQL、SQLite、SQL Server、Oracle に対応したサンプルデータベース、SQL 文をそれぞれ収録してあります。こちらにも活用してください。

MySQL 5.6 (Windows 版) のインストール方法

MySQL 5.6 には、「インストーラを含まないもの」と「インストーラを含むもの」という複数の配布パッケージが用意されていますが、本書ではより簡単に導入ができるインストーラ付きのパッケージ (msi 形式) を採用することにします。

STEP 1 MySQL のインストールファイル mysql-installer-community-5.6.10.0.msi をダウンロードする

MySQL の最新バージョンは、以下の URL から入手できます。

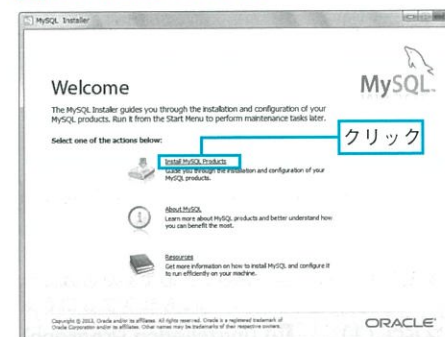
<http://www.mysql.com/downloads/mysql/>

本書では執筆時点での最新版である 5.6.10.0 を前提に解説を進めますが、MySQL は日々精力的にセキュリティ・フィックスや機能改善が行われています。学習にあたっては、その時々 5.6.x 系の最新安定版を使用することをお勧めします。

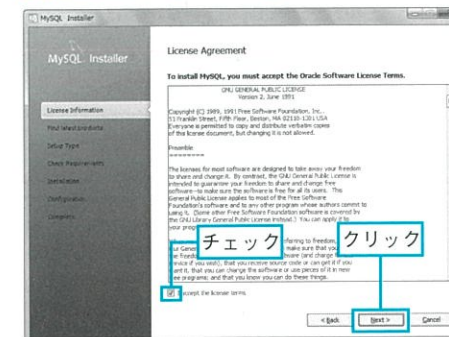
STEP 2 インストール・ウィザードを開始する

ダウンロードした mysql-installer-community-5.6.10.0.msi をダブルクリックし、インストールを開始します。

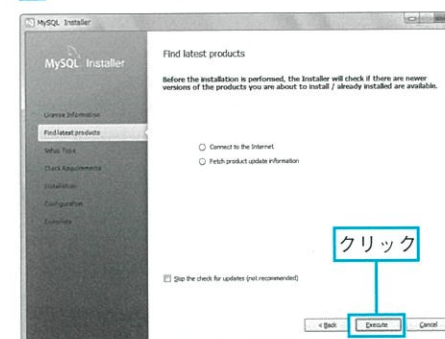
1 [Welcome] ウィンドウ



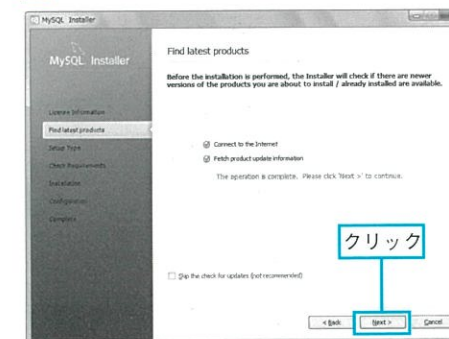
2 [License Agreement] ウィンドウ



3 [Find latest products] ウィンドウ (1)

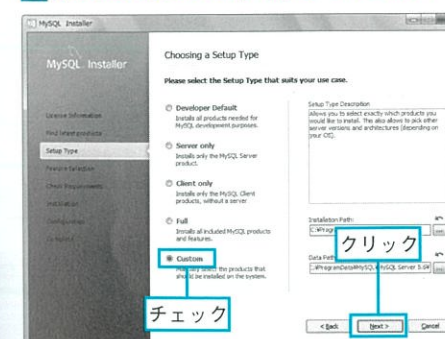


4 [Find latest products] ウィンドウ (2)



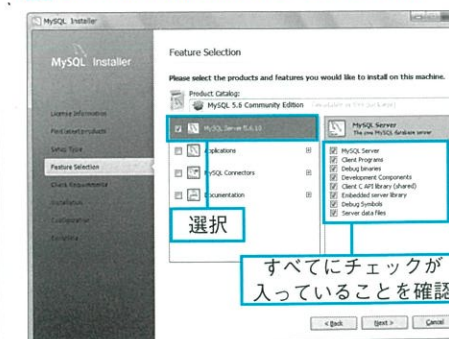
インストールを開始

5 [Choosing a Setup Type] ウィンドウ



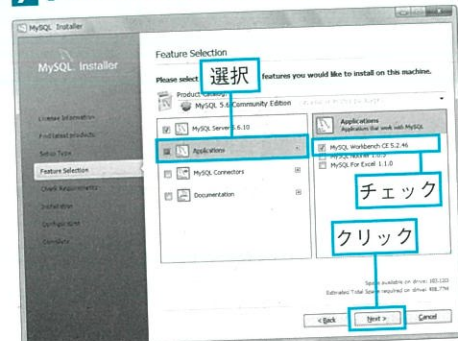
本書では、MySQL Workbench (P.27 参照) を一緒に入れるため [Custom] を選択しますが、MySQL Workbench が不要な場合は [Server only] を選択してください。

6 [Feature Selection] ウィンドウ (1)



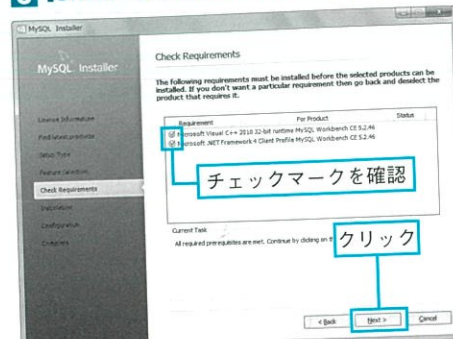
インストールするものを選択します。

7 [Feature Selection] ウィンドウ (2)



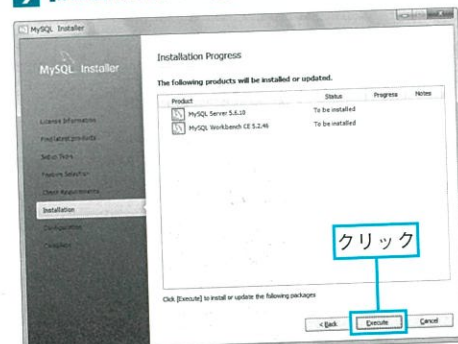
MySQL Workbench をチェックします。そのほか、ドキュメントやサンプルも一緒に入れたい場合は、左枠の [Documentation] にもチェックを入れてください。

8 [Check Requirements] ウィンドウ



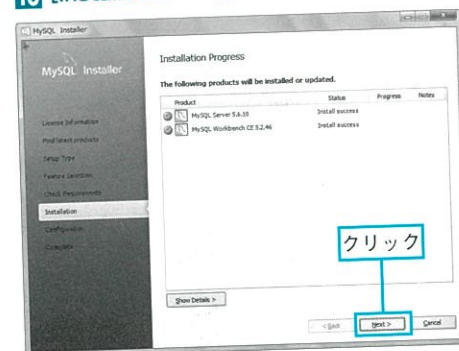
ソフトウェア名の左側にチェックマークが表示されていない場合は、右下に [Execute] ボタンが表示されるのでクリックして、必要なソフトウェアをインストールしてください。

9 [Installation Progress] ウィンドウ (1)

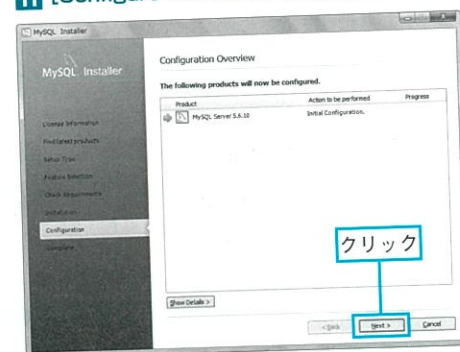


インストールを開始します。

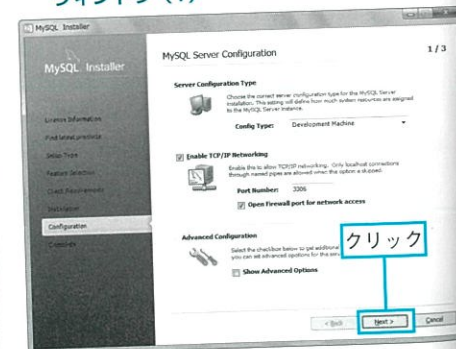
10 [Installation Progress] ウィンドウ (2)



11 [Configure Overview] ウィンドウ



12 [MySQL Server Configuration] ウィンドウ (1)

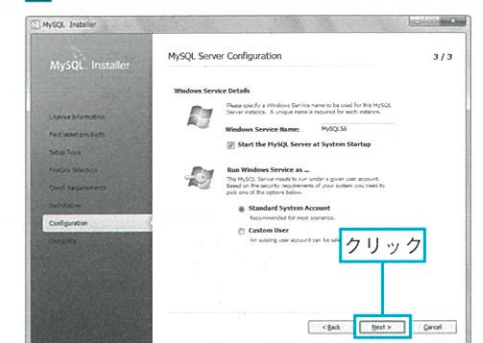


13 [MySQL Server Configuration] ウィンドウ (2)

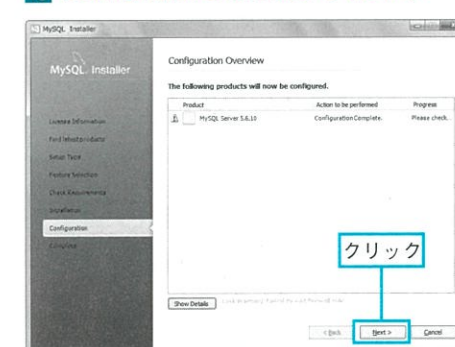


管理者パスワードを設定します。ここでは便宜的に root としておきますが、Weak (弱いパスワード) という警告が表示されます。通常は第3者が想定しにくいような強いパスワードを指定するようにしてください。

14 [MySQL Server Configuration] ウィンドウ (3)

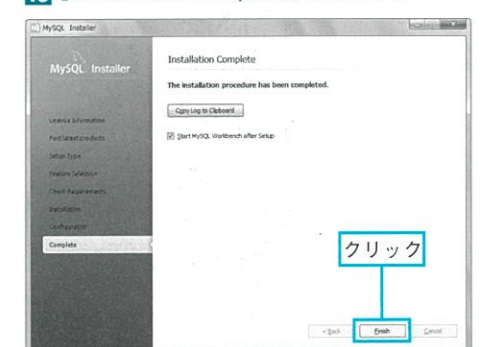


15 [Configuration Overview] ウィンドウ



ファイアウォールに関する警告が表示される場合も、学習には差し支えないため、このまま進めます。

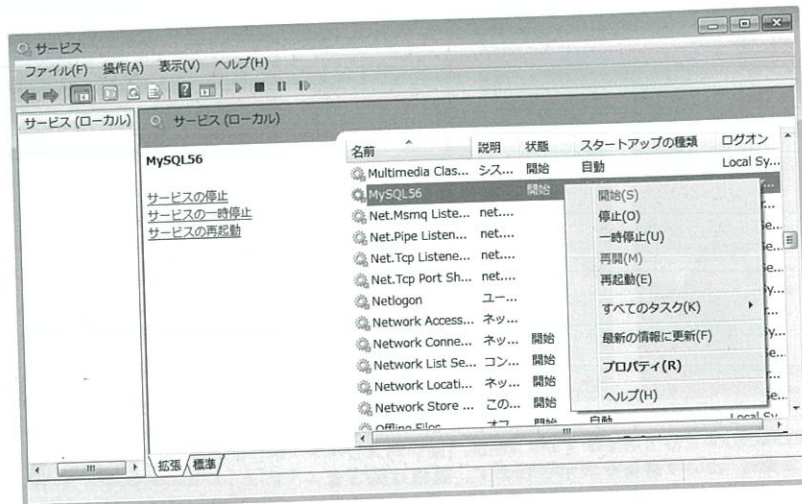
16 [Installation Complete] ウィンドウ



[Start MySQL Workbench after Setup] にチェックを入れておくと、この後自動で MySQL Workbench が起動します。

STEP 3 MySQL を起動する

インストールが終了すると、MySQL は自動で起動しています。実際に MySQL が起動していることを確認してみましょう。コントロールパネルから[システムとセキュリティ] - [管理ツール] - [サービス] を選択して、[サービス] ウィンドウが起動します。



[サービス] ウィンドウ

[名前] 覧から [MySQL56] を見つけたら、[状態] 欄が [開始] と表示されていることを確認します。[状態] 欄が空白の場合、MySQL は停止しているので、[MySQL56] を右クリックし、表示されたコンテキストメニューから [開始] を選択してください。MySQL を再起動/停止したい場合は、コンテキストメニューから [停止] / [再起動] を選択します。

STEP 4 コマンドから MySQL の動作を確認する

コマンドからも MySQL の起動を確認してみましょう。コマンドプロンプトを起動し、以下のように入力してみてください。パスワードを聞かれますので、先ほど設定した「root」を入力します。

```
> mysqladmin -u root -p ping
Enter password: **** ← パスワードを入力
mysql is alive
```

上のように、「mysql is alive」と返されれば、MySQL は正常に動作しています。

NOTE

MySQL のコマンドを実行する場合、「C:\Program Files\MySQL\MySQL Server 5.6\bin」へのパス (環境変数 PATH) を設定しておくことで、環境変数 PATH を設定しておくことで、「> mysqladmin」のようにコマンド名のみでコマンドを実行することができます。

もしも PATH を設定していない場合には、「> "C:\Program Files\MySQL\MySQL

Server 5.6\bin\mysqladmin"] のように絶対パスで指定する必要があります。以降でも同様です。

環境設定を設定する方法は、使用しているプラットフォームによって異なりますが、たとえば Windows 7 ならば [コントロールパネル] - [システムとセキュリティ] - [システム] - [システムの詳細設定] - [環境変数] ボタンから行うことができます。

新規に環境変数を作成する場合は [新規] ボタンをクリックし、既に環境変数が存



[環境変数] ダイアログ

在する場合には [編集] ボタンをクリックしてください。なお、環境変数 PATH を編集する場合には、既存の値を削除しないように、セミコロン (;) 区切りで後ろにつなげてください。

(既存の値) ;C:\Program Files\MySQL\MySQL Server 5.6\bin

STEP 5 学習用のデータベースを展開する

本書で使用するデータベースを MySQL サーバの workbook データベースに展開します。

「c:\」に配布サンプル配下の /database/mysql/workbook.sql をコピーした上で、コマンドプロンプトから以下のコマンドを入力します。

```
> mysqladmin -u root -p CREATE workbook
password: **** ← パスワードを入力
> mysql -u root -p workbook < c:\workbook.sql
password: **** ← パスワードを入力
```

ただし、データベースの展開に失敗した、データを初期状態に戻したい、などの理由で、データベースを展開しないおす場合には、あらかじめ workbook データベースを破棄しておく必要があります。データベースを破棄するには、以下のようにします。

```
> mysqladmin -u root -p DROP workbook
password: **** ← パスワードを入力
```


破棄してもよいか確認されますので、「y」と入力します。

MySQL monitor の利用方法

MySQL データベースを操作するにはさまざまな方法がありますが、もっともシンプルなのは、MySQL に標準で添付されている MySQL クライアント (MySQL monitor) を利用することです。

MySQL monitor は、コマンドプロンプト上で動作する **CUI** (Character User Interface) ベースのツールで、ユーザはプロンプト経由で SQL 命令を発行したり、その結果を確認したりすることができます。MySQL monitor の使い方に特に難しい点はありませんが、学習を始めるにあたって、基本的な操作方法だけを紹介しておきます。

NOTE

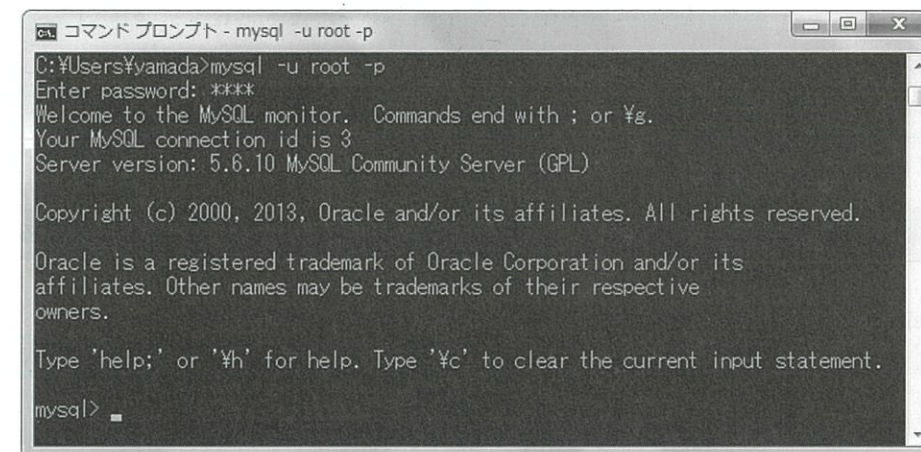
MySQL monitor は、スタートメニューから [すべてのプログラム] - [MySQL] - [MySQL Server 5.6] - [MySQL 5.6 Command Line Client] から起動することも可能です。起動するとすぐにパスワードを聞かれますので、root ユーザのパスワードを入力してログインしてください。

STEP 1 MySQL monitor を起動する

MySQL monitor は、コマンドプロンプトから以下の形式で起動します。

```
> mysql -u ユーザ名 -p
```

-u はユーザ名を、-p はパスワードを指定するためのオプションです。-u、-p オプションをつけて MySQL monitor を起動した場合、パスワードの入力を求められますので、ユーザ名に対応した正しいパスワードを入力してください。



MySQL monitor の起動

STEP 2 データベースを指定する

プロンプトが「>」から「mysql>」に変わったら、MySQL monitor の起動は成功です。それでは、次に、使用するデータベースを指定してみましょう。

```
mysql> USE workbook;
Database changed
```

USE 命令で workbook データベースに移動します。これで、以降に発行される SQL 命令はすべて workbook データベース上で実行されます。

STEP 3 データベースの内容を確認する

データベースの内容を確認するには、**SHOW** 命令を使用します。以下では、データベースに含まれるテーブルの一覧をリストしてみます。

```
mysql> SHOW TABLES;

+-----+
| Tables_in_workbook |
+-----+
| access_log          |
| author              |
| author_books        |
| books               |
| ... 中略 ...        |
| shop                |
| time_card           |
```



```
| usr |
+-----+
18 rows in set (0.00 sec)
```

SHOW TABLES 命令は、現在選択中のデータベース上に存在するテーブル名を表示します。ちなみに、個別のテーブルに属するカラムの一覧を表示するならば、**SHOW FIELDS** 命令を使用します。

```
mysql> SHOW FIELDS FROM books;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| isbn   | char(17) | NO | PRI | NULL | |
| title  | varchar(255) | YES | | NULL | |
| price  | int(11) | YES | | NULL | |
| publish | varchar(30) | YES | | NULL | |
| publish_date | date | YES | | NULL | |
| category_id | char(5) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.03 sec)
```

STEP 4 データベース用の新規ユーザを設定する

昨今の厳しいセキュリティ事情を考えれば、オールマイティなユーザである“root”ユーザを日常のデータベースアクセスで使用することは好ましくありません。本書では、学習用の workbook データベースを利用するための専用ユーザとして“workusr”(パスワード: workpass) を用意しておくことにします。新規ユーザの追加は、以下のようになります。

```
mysql > GRANT ALL PRIVILEGES ON workbook.* TO workusr@localhost
IDENTIFIED BY 'workpass'; (注: 1 行で入力してください)
```

上では、workbook データベースにフルアクセス (ALL PRIVILEGES) 権限を持つ workusr ユーザを新規生成しています。

もちろん、実際のデータベース運用では、用途に応じて、データ更新用のユーザ、検索用のユーザなど、より限定した権限のユーザを複数に分けて用意するべきですが、本書では説明簡略化のために割愛します。

今後、MySQL monitor を起動する際には、ここで作成した workusr ユーザを利用してください。

STEP 5 MySQL monitor を終了する

その他、同様の要領で MySQL monitor 上で SQL 命令を発行することが可能ですが、それらは次章以降で改めてみていくことにしましょう。

最後に、作業を終えた後で、MySQL monitor を終了します。

```
mysql > exit;
```

「mysql >」プロンプトが消え、通常のプロンプトに戻れば成功です。MySQL monitor は終了しました。

補足 データベースのクライアントアプリケーション

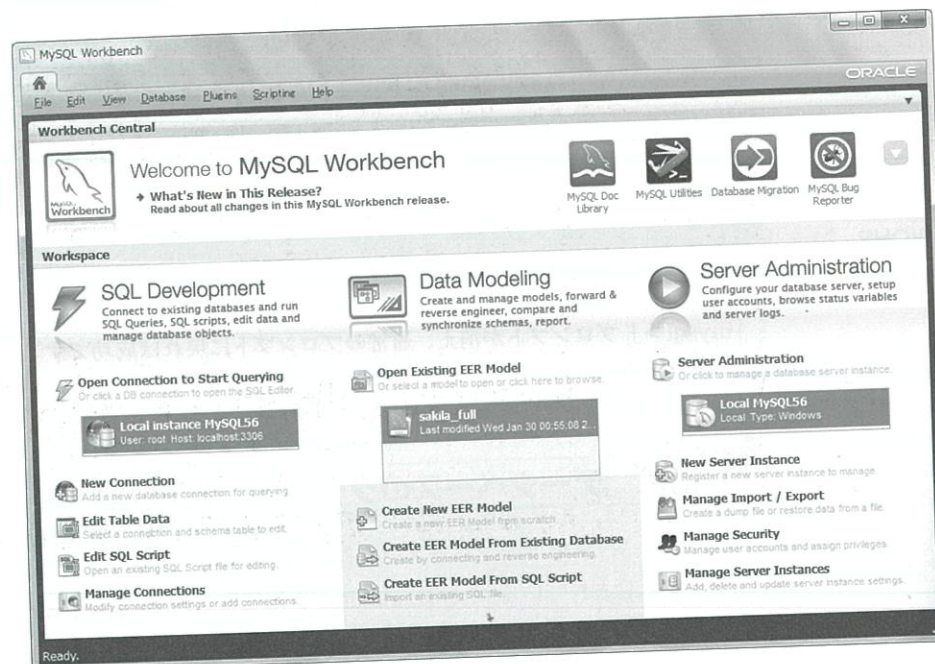
本書では、MySQL データベースを操作するクライアントアプリケーションとして、標準パッケージに含まれている MySQL monitor を使用しています。MySQL monitor は、利用にあたって MySQL の他に特別なソフトウェアのインストールが必要ない、軽量なアプリケーションなので手軽に利用できる、というメリットがあります。

しかし、CUI ベースであるため、テーブルの中身を確認するだけでもいちいち SQL 命令を発行しなければならない、文字コードの設定によってはクライアント側の設定を行わないと文字化けしてしまう、などのデメリットもあります。ある程度、MySQL を使い込んできて、MySQL monitor では面倒くさいと感じ始めたら、GUI ベースのクライアントアプリケーションを導入してみるのも良いかもしれません。

たとえば著者のおすすめとしては、MySQL Workbench があります。MySQL Workbench (<http://dev.mysql.com/downloads/workbench/>) は、GUI ベースの統合管理ツールです。公式サイトで提供されているという安心感もさることながら、データベースの設計、作成、管理などさまざまな用途に対応している高機能性が特長です。

本書の手順に沿っていけば、MySQL サーバと一緒にすでにインストールしています。スタートメニューから [すべてのプログラム] - [MySQL] - [MySQL Workbench 5.2 CE] で起動できます。

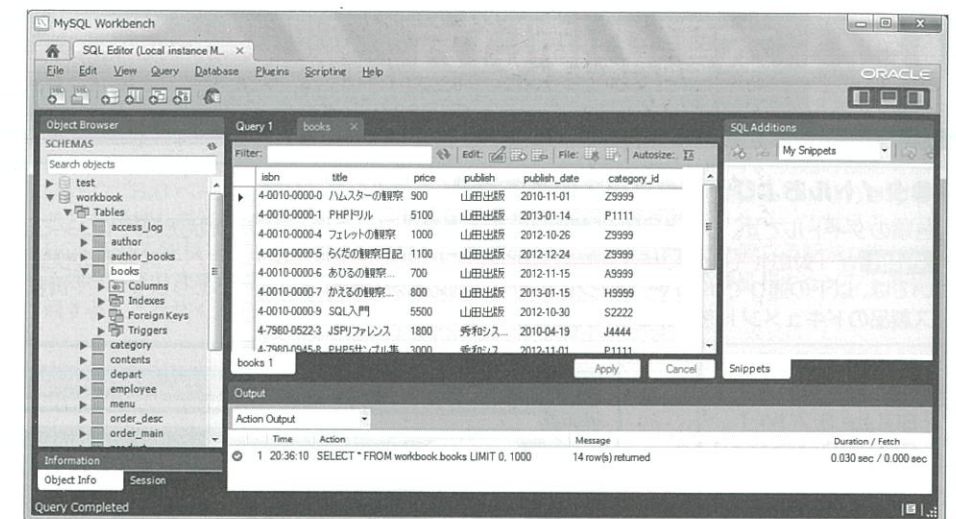
0 イントロダクション



MySQL Workbench

Workbench で SQL を編集／実行するならば、SQL Editor を利用します。SQL Editor は、以下のような特長を持っており、MySQL monitor に較べると格段に操作が容易になっています。

- SQL クエリをハイライト表示できる
- テーブルの内容を GUI ベースで確認できる
- SQL Snippet でよく使うクエリを簡単に再利用できる



SQL Editor

NOTE

MySQL で利用可能なクライアントアプリケーションは、MySQL Workbench だけではありません。また、他のデータベースにも、さまざまなクライアントアプリケーションが用意されています。以下に、各データベースで使える主なクライアントアプリケーションをまとめておきます。

●各データベースで使える主な GUI ツール

データベース	ソフトウェア名	URL
概要		
MySQL	phpMyAdmin	http://www.phpmyadmin.net/
MySQL 操作／管理用の Web アプリケーション。動作には PHP が必要		
PostgreSQL	pgAdmin	http://www.pgadmin.org/
PostgreSQL 操作／管理用のツール。PostgreSQL インストール時に一緒にインストールされる		
SQLite	SQLite Manager	https://addons.mozilla.org/ja/firefox/addon/sqlite-manager/
SQLite 操作用のツール。Firefox のアドオンとして提供		
Oracle	Oracle SQL Developer	http://www.oracle.com/technetwork/jp/developer-tools/sql-developer/
Oracle 操作／管理用のツール		
SQL Server	SQL Server Management Studio	http://msdn.microsoft.com/ja-jp/library/hh213248.aspx
SQL Server 操作／管理用のツール。無償版の SQL Server Management Studio Express もある		

0-4 本書の使い方

●タイトルおよびデータベースの対応状況

各項目のタイトルです。タイトルの下部には、本書で扱うデータベースの対応状況をアイコンで示しています。
MySQL が「対応」、MySQL が「非対応」を表しています。なお、各データベース製品の検証バージョンについては、以下の通りです。バージョンによっては対応状況が異なる可能性もありますので、詳細は各データベース製品のドキュメントを参照してください。

- ・ MySQL 5.6.10
- ・ PostgreSQL 9.2
- ・ SQLite 3.7.14.1
- ・ Oracle Database 11g Express Edition
- ・ SQL Server 2012 Express Edition

なお、PostgreSQL、SQLite のインストール手順などについては、著者サポートサイト「サーバサイド技術の学び舎-WINGS (<http://www.wings.msn.to/>)」より「サーバサイド環境構築設定」を参照してください。

●例題

本書では、テーマごとに1つの例題を紹介しています。例題と回答例から、これから学習するテーマの概要と具体的な動作イメージを把握してみましょう。ただし、結果はデータベース内のデータやその他条件によって異なる可能性があります。また、例題の SQL 命令は MySQL 5.6 で動作を確認しています。その他のデータベースでは記法が異なる場合もありますので、注意してください。末尾に SQL の動作イメージを表していますが、これはあくまで動作の概念的なイメージを表したもので、必ずしもデータベースが同様の操作を行っていることを表すものではありません。

1-1 テーブルからすべての列を取り出し SELECT 命令

MySQL PostgreSQL SQLite SQL Server Oracle



手始めに、まずはもっとも基本的なテーブルの検索を行ってみましょう。

アンケート回答テーブル (quest) からすべてのデータを取得し、その結果を一覧表に出力しましょう。

テーブル内のデータを取得するのは、SELECT 命令の役割です。

```
> SELECT
> *
> FROM
> quest
> ;
```

id	name	name_kana	sex	prefecture	age	answer1	answer2	answered
1	山田太郎	ヤマダタロウ	男	東京都	30	3	重宝しています。	2012-10-10 10:11:12
2	井上茉莉	イノウエマリ	女	神奈川県	25	2	面白いです。	2012-10-16 07:15:14
3	上原運	ウエハラハルカ	女	茨城県	26	1	ちょっと難しいです。	2012-10-18 20:11:12
4	江本修子	エモトシュウコ	女	東京都	32	3	次回作に期待しています。	2012-10-24 21:14:18
5	小野博美	オノヒロミ	女	神奈川県	40	2	わかりやすいです。	2012-10-29 21:18:17
6	河合太郎	カワイタロウ	男	東京都	22	1	絵が少ない。	2012-11-01 20:10:14
7	矢口一樹	ヤグチカズキ	男	茨城県	26	2	NULL	2012-11-06 20:11:12
8	有木守	アリキマズル	男	千葉県	35	3	買ってよかった。	2012-11-10 21:14:18
9	吉岡純	ヨシオカツシ	男	埼玉県	18	3	NULL	2012-11-21 14:15:15
10	和田健	ワダケン	男	東京都	41	1	NULL	2012-11-26 21:18:17

16 rows in set (0.00 sec)

アンケート回答テーブル (quest)

id	name	sex	answered
1	山田太郎	ヤマダタロウ	2012-10-10 10:11:12
2	井上茉莉	イノウエマリ	2012-10-16 07:15:14
3	上原運	ウエハラハルカ	2012-10-18 20:11:12
4	江本修子	エモトシュウコ	2012-10-24 21:14:18
5	小野博美	オノヒロミ	2012-10-29 21:18:17

(次ページへ)

●構文を理解しよう

例題で使用している SQL 命令の基本的な構文、記述する際のポイントや注意点を解説しています。

構文中に含まれる「[]」はそのキーワードが省略可能であることを表しています。

また、ここではデータベース製品ごとの方言についても紹介しています。例題の回答では MySQL 5.6 ベースで記述していますので、その他のデータベースを使用している方は必ずここを確認してください。ただし、あくまで本書の目的は個々のデータベース製品ごとの SQL 命令を総括することではありません。本書で紹介した記法以外にも異なる記法が存在する場合もありますので、データベースごとの記法については、個々の製品ドキュメントを参照することを強くお勧めします。

1-1 テーブルからすべての列を取り出したい

(前ページから)

* (アスタリスク)
すべての行を無条件に選択

id	name	sex	answered
1	山田太郎	ヤマダタロウ	2012-10-10 10:11:12
2	井上茉莉	イノウエマリ	2012-10-16 07:15:14
3	上原運	ウエハラハルカ	2012-10-18 20:11:12
4	江本修子	エモトシュウコ	2012-10-24 21:14:18
5	小野博美	オノヒロミ	2012-10-29 21:18:17

●構文を理解しよう

SELECT 命令の基本的な構文は、以下の通りです。

構文 SELECT 命令①

SELECT 列名 FROM テーブル名 ;

選択しない 何を? どこから? このテーブルから

これを単純に日本語に訳してみると、「<テーブル名>から<列名>を選択しない」というわけです。この場合、テーブル名は quest です。

例題の A では列名として「*」を指定していますが、これは「すべての列名」を表します。とりあえずテーブルに含まれるすべての列 (フィールド) を取り出したいという場合には、アスタリスク (*) を指定することでテーブル内にどのような情報が含まれているかなどをまったく意識することなく、データを取り出すことができます。

●SQL 命令を記述してみよう

SELECT 命令は SQL 命令のさまざまな命令の中でも、最もよく利用する命令の1つです。また、豊富に用意されたオプションを組み合わせることで、本項で扱うようなシナリオな問い合わせから複雑な問い合わせまで、実に多彩な表現を実現できます。

複雑な問い合わせになっても迷わないように、ここでは記述の上での骨組みを理解していくことにしましょう。

STEP 1 なにをしたいの?

SQL 命令といっても、扱うことができる命令はさまざまです。今後登場しますが、INSERT (挿入しない)、UPDATE (更新しない)、DELETE (削除しない) ... などなど。まずはデータベースに対して「なにをしたいのか」を伝える必要があります。

●SQL 命令を記述してみよう

本書では、SQL 命令を実際に手を動かしながら書いていくことを主目的としています。単に構文を記憶するだけではなく、手順通りに記述していくことで、SQL の構造を理解するようにしてください。なお、この STEP に沿って記述する際は、MySQL monitor で入力する前にメモ帳などを起動して手順に沿って入力し、【完成】したら、MySQL monitor に記述した SQL 命令をコピー&ペーストするようにしましょう。

後半では、「復習しよう」という項が登場しますが、ここではそこまでの手順に従って、自分で SQL 命令を書き込んでみましょう。

0 イントロダクション

マスタ・プラクティス

本テーマに関わる練習問題です。基礎的な空欄補足問題から記述式、誤りを見つける問題まで多岐にわたりますが、ただ単に回答を導くことだけに終始しないでください。まずは本文の「SQL を記述してみよう」で紹介した手順に従って、自分で正しい SQL 命令を記述してください。その上で問題に対する解を導くようにすることで、より一層実力が身につくはずです。本書には空白を多く用意しています。本書を黒く塗りつぶすまで、書き込んでみましょう。解答は巻末の解答集をご覧ください。

●注意

SQL 命令は改行を意識しません。本文の解説部分では、句の区切り目で改行を入れて、視覚的に構造が分かりやすくなるように示しています。このため、以下のような穴埋め問題の場合、①に入る命令文が巻末の解答で1行で表示されているとは限りません。設問内容をよく読んで、正しい SQL を記述しましょう。

問 1

書籍情報テーブル (books) からすべての列を取り出してみましょう。以下の空欄を埋めて、SQL 命令を完成させてください。

```

①
*
FROM
②
;
```

NOTE

本文では紹介しきれなかったテクニックや、その他、知っておいた方が良い豆知識を紹介します。

配布サンプルについて

本書で紹介するデータベース、及び、ドリルで使用している SQL 命令は、以下のページからダウンロードできます。

著者サイト「サーバサイド技術の学び舎 - WINGS」 - [総合 FAQ/ 訂正 & ダウンロード]
<http://www.wings.msn.to/index.php/-/B-11/>

ドリルを開始するにあたっては、それぞれのデータベース用に用意されたサンプルデータベース (workbook.sql、workbook.mdf、workbook.sqlite) を展開、または配置してください。

なお、既に説明したように、SQL の構文はデータベース製品によって、微妙に異なる場合があります。主要な相違点については本文内で紹介していますが、細かい方言などについては、配布サンプル内の .sql ファイルを確認することを強くお勧めします。この .sql ファイル内には、例題で使用している SQL 文のほか、練習問題の解答例も記述してあります。

/database

/MySQL	chap1.sql	(Chapter1 の SQL)
	chap2.sql	(Chapter2 の SQL)
	chap3.sql	(Chapter3 の SQL)
	chap4.sql	(Chapter4 の SQL)
	workbook.sql	(サンプルデータベース)
/PostgreSQL	chap1.sql	(Chapter1 の SQL)
	chap2.sql	(Chapter2 の SQL)
	chap3.sql	(Chapter3 の SQL)
	chap4.sql	(Chapter4 の SQL)
	workbook.sql	(サンプルデータベース)
/Oracle	chap1.sql	(Chapter1 の SQL)
	chap2.sql	(Chapter2 の SQL)
	chap3.sql	(Chapter3 の SQL)
	chap4.sql	(Chapter4 の SQL)
	workbook.sql	(サンプルデータベース)
/SQLServer	chap1.sql	(Chapter1 の SQL)
	chap2.sql	(Chapter2 の SQL)
	chap3.sql	(Chapter3 の SQL)
	chap4.sql	(Chapter4 の SQL)
	workbook.mdf	(サンプルデータベース)
/SQLite	chap1.sql	(Chapter1 の SQL)
	chap2.sql	(Chapter2 の SQL)
	chap3.sql	(Chapter3 の SQL)
	chap4.sql	(Chapter4 の SQL)
	workbook.sqlite	(サンプルデータベース)

0 インTRODクシヨン

 本書で使っているサンプルデータベース

本書で利用しているサンプルデータベースには、以下のようなテーブルを含んでいるものとします。それぞれのテーブルは、以下のようなアプリケーションで利用されることを想定しています。具体的なアプリケーションと業務をイメージしながら、データを操作することで、より理解も深まることを期待しています。

- ・サイトからのアンケート収集
- ・図書館の書籍貸し出し管理
- ・企業における社員の勤怠管理
- ・チェーン店を持つ小売店の売り上げ管理
- ・インターネット上で商品売買を行うサイト
- ・Web ページ上のアクセスログ管理
- ・書籍のようなひとつつらりのコンテンツを管理

これらのテーブルはいずれも実業務で実際に扱うものに比べると、大幅に簡略化されていたり、導入ドリルという本書の目的から、あえて正規化を行っていない（あるいは一部を非正規化している）ものもありますので、ご了承ください。

アスタリスク（*）がついている項目は主キーであることを、項目名が斜体である列は Null 値を許さないことを、それぞれ表します。

 共通テーブル

「図書貸し出し」「電子商店街」アプリケーションで共通して利用しているテーブルです。ユーザに関する情報を管理します。

●ユーザテーブル (usr)

列名	データ型	概要
<i>user_id</i> *	CHAR(7)	利用者コード
<i>l_name</i>	VARCHAR(20)	利用者名 (氏)
<i>f_name</i>	VARCHAR(20)	利用者名 (名)
<i>l_name_kana</i>	VARCHAR(100)	利用者名 (氏カナ)
<i>f_name_kana</i>	VARCHAR(100)	利用者名 (名カナ)
<i>prefecture</i>	VARCHAR(15)	住所 (都道府県名)
<i>city</i>	VARCHAR(20)	住所 (市町村名)
<i>o_address</i>	VARCHAR(100)	住所 (その他)
<i>tel</i>	VARCHAR(20)	電話番号
<i>email</i>	VARCHAR(255)	E-Mail アドレス

 アンケート集計アプリケーション

Web ページから収集したアンケートの回答結果を管理します。回答結果を格納するための quest テーブルが用意されています。

●アンケート回答テーブル (quest)

列名	データ型	概要
<i>id</i> *	INT AUTO_INCREMENT	回答コード
<i>name</i>	VARCHAR(100)	回答者名
<i>name_kana</i>	VARCHAR(255)	回答者名 (カナ)
<i>sex</i>	VARCHAR(5)	性別 (男 女)
<i>prefecture</i>	VARCHAR(10)	都道府県
<i>age</i>	INT	年齢 (デフォルト 0)
<i>answer1</i>	INT	本書の評価 (3: ためになった, 2: 普通, 1: 役に立たない)
<i>answer2</i>	TEXT	本書の感想
<i>answered</i>	DATETIME	回答日時

 図書館貸し出しアプリケーション

図書館などで図書の貸し出し管理を行うためのアプリケーションを想定しています。書籍情報テーブルが書籍に関する情報を管理するマスターテーブルです。著者情報テーブルは著者に関する情報を管理するテーブルで、著者-書籍情報テーブルを介して書籍情報テーブルと関連付けられています (1 冊の書籍が複数の著者を持つ場合を想定しているわけです)。

貸し出し記録テーブルは、書籍の貸し出しごとに蓄積されていくテーブルです。返却後も returned 列が 1 となるだけで貸し出し情報は残りますので、過去の貸し出し履歴を管理するテーブルにもなっています。

0 イン트로ダクション

●書籍情報テーブル (books)

列名	データ型	概要
isbn *	CHAR(17)	ISBN コード
title	VARCHAR(255)	書名
price	INT	価格
publish	VARCHAR(30)	出版社
publish_date	DATE	刊行日
category_id	CHAR(5)	分類 ID

●著者-書籍情報テーブル (author_books)

列名	データ型	概要
isbn *	CHAR(13)	ISBN コード
author_id *	CHAR(5)	著者 ID

●著者情報テーブル (author)

列名	データ型	概要
author_id *	CHAR(5)	著者 ID
name	VARCHAR(30)	著者名
name_kana	VARCHAR(100)	著者名カナ
birth	DATE	生年月日

●書籍分類テーブル (category)

列名	データ型	概要
category_id *	CHAR(5)	分類 ID
category_name	VARCHAR(50)	分類名

●ユーザーテーブル (usr)

列名	データ型	概要
user_id *	CHAR(7)	利用者コード
l_name	VARCHAR(20)	利用者名 (氏)
f_name	VARCHAR(20)	利用者名 (名)
l_name_kana	VARCHAR(100)	利用者名 (氏カナ)
f_name_kana	VARCHAR(100)	利用者名 (名カナ)
prefecture	VARCHAR(15)	住所 (都道府県名)
city	VARCHAR(20)	住所 (市町村名)
o_address	VARCHAR(100)	住所 (その他)
tel	VARCHAR(20)	電話番号
email	VARCHAR(255)	E-Mail アドレス

●貸し出し記録テーブル (rental)

列名	データ型	概要
id *	INT AUTO_INCREMENT	貸し出し ID
user_id	CHAR(7)	利用者コード
isbn	CHAR(13)	ISBN コード
rental_date	DATE	貸出日
returned	SMALLINT	返却済みかどうか (1: 返却済み 0: 貸し出し中)。デフォルト 0

●社員テーブル (employee)

列名	データ型	概要
s_id *	CHAR(7)	社員コード
l_name	VARCHAR(20)	社員名 (氏)
f_name	VARCHAR(20)	社員名 (名)
l_name_kana	VARCHAR(100)	社員名 (氏カナ)
f_name_kana	VARCHAR(100)	社員名 (名カナ)
sex	SMALLINT	性別 (1: 男、2: 女、3: その他)
class	VARCHAR(20)	役職 (部長、課長、主任、担当、アシスタント)
depart_id	CHAR(3)	所属部署コード
b_id	CHAR(7)	上司コード (社員コードに対応。上司がいない社員は NULL)
last_update	DATE	最終更新日
retired	SMALLINT	退職済みか (1: 退職済み 0: 在職)

●タイムカードテーブル (time_card)

列名	データ型	概要
s_id *	CHAR(7)	社員コード
r_date *	DATE	日付
work_time	DECIMAL	勤務時間

●所属部署テーブル (depart)

列名	データ型	概要
depart_id *	CHAR(3)	所属部署コード
depart_name	VARCHAR(20)	所属部署名

勤怠管理アプリケーション

会社内の勤怠管理を行うためのアプリケーションを想定しています。

社員テーブルは社員とその所属部署、上司部下の関係を表しています。

タイムカードテーブルは、社員、日ごとの勤務時間を管理するためのテーブルです。

ただし、基本的に勤務時間管理を行うのは、主任以下の役職のみとし、課長、部長についてはデータ管理はしていないものとします。

売り上げ管理アプリケーション

配下に支店を持つ小売販売業における売り上げ管理を想定しています。

店舗テーブルでは店舗の情報を、月間売り上げテーブルでは店舗、月ごとの売上高を管理しているものとします。店舗ごとにデータの入力は任されているものとし、店舗によっては、すべての月のデータが揃っていない場合もあります。

●店舗テーブル (shop)

列名	データ型	概要
s_id *	CHAR(5)	店舗コード
s_name	VARCHAR(30)	店舗名

●月間売り上げテーブル (sales)

列名	データ型	概要
s_id *	CHAR(5)	店舗コード
s_date *	CHAR(7)	売り上げ対象年月 (2013-12 など)
s_value	INT	売上高 (千円)

0 イントロダクション

電子商店街アプリケーション

インターネット上で公開された電子商店街（EC サイト）を想定しています。

商品テーブルでは、サイト上から購入できる商品の情報を管理しています。

注文情報を表すのは、注文書テーブルと注文明細テーブルの役割です。注文書テーブルは、購入者や発注／納品日など注文の中核となる情報を表し、購入した個々の商品に関する明細情報は注文明細テーブルで管理されます。

●注文書テーブル (order_main)

列名	データ型	概要
po_id *	INT AUTO_INCREMENT	注文番号
user_id	CHAR(7)	利用者コード
order_date	DATE	発注日
delivery_date	DATE	納品日（本フィールドが NULL の場合、注文は未納）

●ユーザーテーブル (usr)

列名	データ型	概要
user_id *	CHAR(7)	利用者コード
l_name	VARCHAR(20)	利用者名（氏）
f_name	VARCHAR(20)	利用者名（名）
l_name_kana	VARCHAR(100)	利用者名（氏カナ）
f_name_kana	VARCHAR(100)	利用者名（名カナ）
prefecture	VARCHAR(15)	住所（都道府県名）
city	VARCHAR(20)	住所（市町村名）
o_address	VARCHAR(100)	住所（その他）
tel	VARCHAR(20)	電話番号
email	VARCHAR(255)	E-Mail アドレス

●注文明細テーブル (order_desc)

列名	データ型	概要
po_id *	INT	注文番号
p_id *	CHAR(10)	商品コード
quantity	INT	購入数

●商品テーブル (product)

列名	データ型	概要
p_id *	CHAR(10)	商品コード
p_name	VARCHAR(255)	商品名
price	INT	単価

サイトアクセスログ管理アプリケーション

Web サイト上のアクセスログ管理を想定しています。

メニューテーブルでは、サイト上で公開している具体的なメニューとその上下階層を表します。アクセス記録テーブルがそれぞれのメニュー（ページ）に対するアクセス情報（リファラや接続元の IP アドレス）を明細管理します。アクセス記録テーブルは、1 アクセスごとに 1 行の情報が追加されていきますので、アクセス数を知るには、このテーブルを集計する必要があります。

●メニューテーブル (menu)

列名	データ型	概要
page_id *	CHAR(5)	メニューコード
title	VARCHAR(100)	メニュー名
parent	CHAR(5)	親メニュー

●アクセス記録テーブル (access_log)

列名	データ型	概要
log_id *	INT	ログ ID
page_id	CHAR(5)	メニューコード
referer	VARCHAR(255)	リンク元の URL
ip_address	CHAR(15)	IP アドレス
access_date	DATETIME	アクセス日時

書籍の目次管理アプリケーション

たとえば、レポートや小説のようにひとつつらりの内容を持つコンテンツを想定してください。書籍目次テーブルは、その個々の目次項目を表すと同時に、項目の前後関係を表わすためのテーブルです。next_id 列と c_id 列とが対応関係にあり、next_id 列には現在のコンテンツの次のコンテンツコードがセットされています。

●書籍目次テーブル (contents)

列名	データ型	概要
c_id *	CHAR(4)	コンテンツコード
c_title	VARCHAR(255)	コンテンツ名
next_id	CHAR(4)	次のコンテンツコード（コンテンツコードに対応）